

Lab8 Report

22300240022 王镜凯

一、背景与目标

本模块实现了编译器后端中的图着色寄存器分配（Register Allocation via Graph Coloring）。其目标是在满足目标机寄存器数量限制的前提下，为程序中变量分配寄存器，并尽可能减少溢出（spill）。

本实现使用的是 **Briggs** 的着色图简化与合并策略，整体流程参考了经典的五步框架：

1. Simplify（简化）
2. Coalesce（合并）
3. Freeze（冻结）
4. Spill（选择溢出）
5. Select & Color（选择与着色）

二、关键数据结构

- `graph`：干涉图，`map<int, set>` 类型，表示变量之间的冲突关系。
- `movePairs`：可优化的 move 指令对。
- `coalescedMoves`：记录哪些节点被合并到某个主节点上。
- `simplifiedNodes`：栈结构，保存简化或溢出节点的顺序。
- `colors`：着色结果，映射每个节点到某个寄存器编号。
- `spilled`：集合，记录最终未能着色的溢出节点。

三、各阶段实现详解

1. Simplify 简化阶段

- 目标**：移除度数（邻接点数量）小于 k 的非机器寄存器节点，并推入 `simplifiedNodes`。
- 逻辑**：遍历图中节点，如果其度数 $< k$ ，且不是 move 节点或机器寄存器，则从图中删除并标记。
- 优势**：简洁直接，符合理论要求。

2. Coalesce 合并阶段

- 策略**：采用 Briggs 的 conservative 合并策略（高度邻居节点数量 $< k$ ）。
- 流程**：
 - 忽略机器寄存器间的 move。
 - 若两点不冲突，统计合并后邻居中度数 $\geq k$ 的数量。

- 安全则将 `v` 合并入 `u`，添加所有边。
- 亮点：合理保留 `move` 合并优化机会，减少实际溢出。

3. Freeze 冻结阶段

- 目标：对于无法安全合并的 `move` 节点，若其度数 $< k$ ，则将其 `move` 指令“冻结”，避免无限循环尝试合并。
- 实现：
 - 遍历图中节点，找出 `move` 节点。
 - 若度数 $< k$ ，则删除所有涉及该节点的 `move`。

处理简单有效，保证后续可以进入 `simplify`。

4. Spill 选择溢出节点

- 策略：选择度数最高的节点作为溢出候选。
- 亮点：
 - 使用 `soft spill` 策略：先假装简化，真实溢出在 `select` 阶段处理。

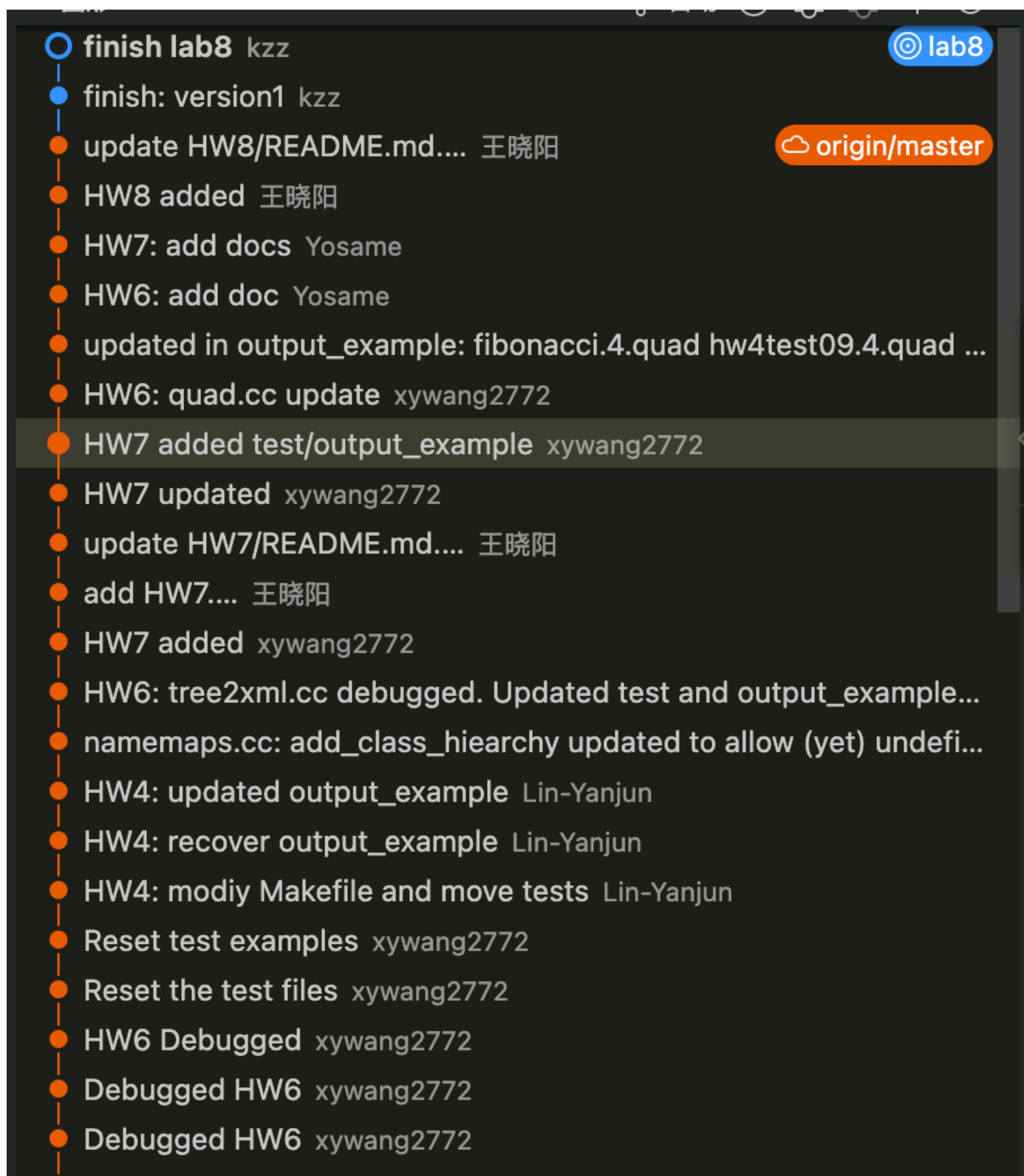
5. Select & Color 着色阶段

- 流程：
 - 从 `simplifiedNodes` 中依次弹出节点。
 - 跳过合并过的节点或机器寄存器。
 - 查找所有邻居的颜色集合，选择一个可用颜色。
 - 若无可用颜色则标记为 `spill`。
 - 最后将主节点的颜色传递给所有合并进来的节点。
- 亮点：
 - 考虑了合并节点的所有邻居，保证颜色合法性。
 - 同时处理了合并节点与主节点之间的冲突避免。
- 检查：
 - `checkColoring()` 应确保最终着色图中无冲突。

四、功能验证与调试策略

- 通过 `#define DEBUG` 和 `DEBUG_OUTPUT` 宏，控制调试信息输出，便于开发调试。
 - 调试输出内容包括：
 - 节点的简化、冻结、溢出过程。
 - 合并尝试与成功情况。
 - 最终着色分配情况。
-

Git 开发图



参考资料

- tiger
- PPT